Lab 3 Report

Quadrotor modelling and geometric trajectory control

Name: _____ Student number: _____

Lab group: _____

Lab date: _____ Report date: 10 October 2026

Summary

This experiment connects rotation representations, quadrotor actuation and geometric control in a ROS trajectory-tracking system. The rotor-allocation analysis shows why reversing two propeller configurations removes one independent control direction. A body-frame quadratic drag model gives an explicit force-compensation term. The implemented controller was evaluated using recorded flight from the official Unity/TESSE simulator on a circular reference of radius 5 m. After the first two reference periods, the vehicle completed 2.376 observed turns with a three-dimensional position RMS error of 0.3063 m and a radial RMS error of 0.0829 m. The error decomposition identifies a persistent height offset as the main contributor. Separate numerical results illustrate how the specified drag compensation reduces tracking error when the plant follows that drag law.

1 Lab Introduction and Team Information

Quadrotor trajectory tracking requires a consistent connection between state coordinates and motor commands. A correct feedback equation can generate the wrong motion if the rotation direction, thrust sign or rotor numbering changes between the model and the simulator. This lab examines that connection through the individual exercises and the ROS controller implementation specified in MIT VNAV Lab 3 [1].

The report follows the three deliverables: quaternion conversions, modelling and control analysis, and geometric control of a simulated UAV. The implemented control loop adapts the geometric approach of Lee et al. [2] to an upward world and body z axis. Its performance is assessed using measured trajectories, a retained RViz recording and numerical checks of the shared controller equations.

Team members and task allocation: _____

2 Individual Work

2.1 Deliverable 1: From attitude messages to control variables

The controller receives the drone's orientation as a quaternion in a ROS message. Its feedback equations need a rotation matrix, while the desired heading is specified by a yaw angle. This exercise explains how to obtain those quantities from the received data.

Three C++ types appear in this process. A `geometry_msgs::Quaternion` carries the four components (x, y, z, w) between ROS nodes. A `tf2::Quaternion` stores the same rotation in a form supported by the tf2 rotation utilities. An `Eigen::Matrix3d` stores the corresponding 3×3 rotation matrix for matrix and vector calculations. Converting between the quaternion and matrix representations preserves the represented orientation; extracting yaw selects its heading component.

A. Reading and writing quaternion data (A1–A3)

For A1, suppose `quat_msg` is the received message and `quat_tf` is an already declared tf2 quaternion. The following call copies the message components into `quat_tf` [3]:

```
tf2::fromMsg(quat_msg, quat_tf);
```

For A2, the reverse operation creates a ROS message from a computed tf2 quaternion. This is useful when an orientation must be published to another node:

```
geometry_msgs::Quaternion quat_msg = tf2::toMsg(quat_tf);
```

These are separate code examples, using the variable names specified in the exercise.

For A3, the scalar component of the quaternion is obtained with:

```
quat_tf.w();
```

A quaternion contains a vector part (x, y, z) and a scalar part w. Reading w returns one stored component. Calculating yaw requires the rotation represented by the quaternion, as shown next.

B. Obtaining heading and a rotation matrix (B1–B2)

For B1, extract yaw from the tf2 quaternion `quat_t` with one function call [4]:

```
double yaw = tf2::getYaw(quat_t);
```

The result is a heading angle in radians. In this lab, the desired-state callback uses this operation to obtain ψ_d from the reference orientation. The controller then uses ψ_d to construct its desired heading direction.

For B2, convert a ROS quaternion message into an Eigen quaternion, then convert that quaternion into a rotation matrix [5]:

```
Eigen::Quaterniond q_eigen;
tf2::fromMsg(quat_msg, q_eigen);
Eigen::Matrix3d R = q_eigen.toRotationMatrix();
```

The first line declares storage; the next two lines perform the two requested conversions. Here the input is assumed to be a unit quaternion. The actual current-state callback checks the input and normalises it before forming R.

For the current attitude in this lab, R maps a body-frame vector to the world frame: $v^w = Rv^b$. Its transpose performs the reverse conversion. The controller therefore computes $\Omega = R^T\Omega_{\text{world}}$ to express the received angular velocity in body coordinates. It also uses R to calculate attitude error and the current thrust direction. Thus yaw extraction supplies the desired heading, while matrix conversion supplies the current attitude needed by the feedback equations.

2.2 Deliverable 2A: Rotor layout and control authority

Use the top view in Figure 1, with x to the right, y upward on the page and z out of the page. Let d be the centre-to-rotor distance, $a = d/\sqrt{2}$, $c_f > 0$ the lift coefficient and $c_d > 0$ the yaw reaction-torque coefficient. The individual exercise numbers the rotor positions as

$$\rho_1 = (-a, a, 0)^T, \quad \rho_2 = (-a, -a, 0)^T, \quad \rho_3 = (a, -a, 0)^T, \quad \rho_4 = (a, a, 0)^T. \tag{1}$$

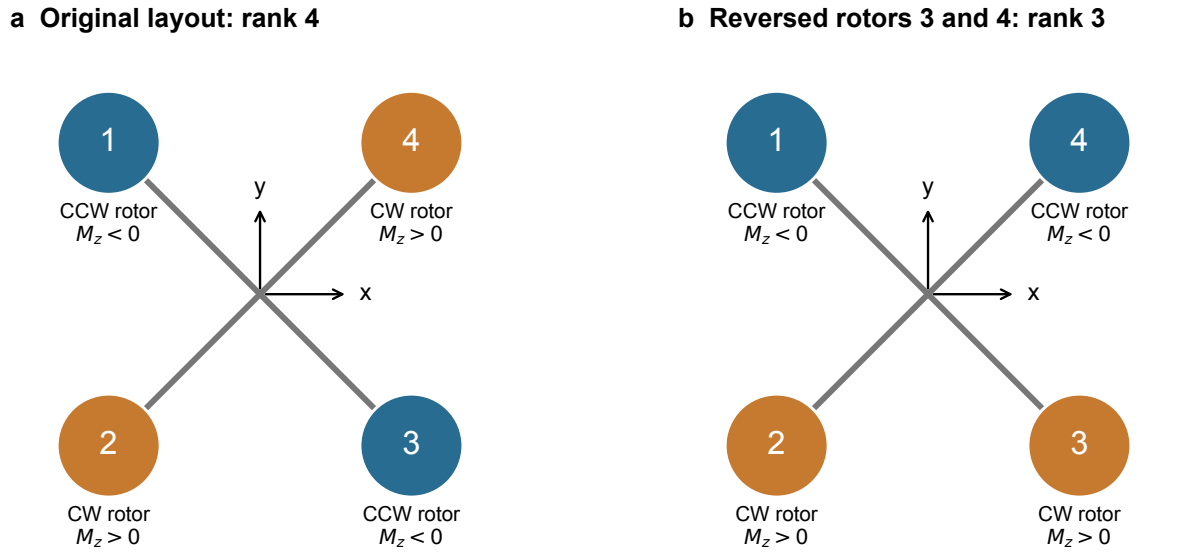


Figure 1: Rotor configurations for the individual exercise. CW and CCW denote rotation viewed from above; the indicated M_z is the resulting body reaction torque. Reversing rotors 3 and 4, with matching propeller types, preserves upward lift and changes the yaw-torque signs. Diagram redrawn from the course configuration [1].

Let $u_i = \omega_i^2 \geq 0$, with ω_i measured in each rotor's configured lifting direction. The force is $f_i = c_f u_i$, and the moment arm gives

$$\rho_i \times (0, 0, f_i)^T = (\rho_{iy} f_i, -\rho_{ix} f_i, 0)^T. \tag{2}$$

For the wrench $w = (f, M_x, M_y, M_z)^T$, configuration (a) therefore has

$$w = F_a u, \qquad F_a = \begin{bmatrix} c_f & c_f & c_f & c_f \\ ac_f & -ac_f & -ac_f & ac_f \\ ac_f & ac_f & -ac_f & -ac_f \\ -c_d & c_d & -c_d & c_d \end{bmatrix}. \tag{3}$$

After division by their nonzero scale factors, the four rows are mutually orthogonal. Consequently,

$$\det F_a = 16a^2 c_f^3 c_d = 8d^2 c_f^3 c_d \neq 0, \qquad \operatorname{rank}(F_a) = 4. \tag{4}$$

All four wrench components can be adjusted independently near a positive-speed hover, within actuator limits.

For configuration (b), only the last row changes:

$$F_b = \begin{bmatrix} c_f & c_f & c_f & c_f \\ ac_f & -ac_f & -ac_f & ac_f \\ ac_f & ac_f & -ac_f & -ac_f \\ -c_d & c_d & c_d & -c_d \end{bmatrix}. \tag{5}$$

The last row equals $-c_d/(ac_f)$ times the second row. The other three rows remain independent, so

$$\boxed{\operatorname{rank}(F_b) = 3, \qquad M_z = -\frac{c_d}{ac_f} M_x.} \tag{6}$$

For example, $F_b(1, -1, 1, -1)^T = 0$. At level hover with zero angular velocity, holding position while starting a pure yaw rotation requires zero roll moment and a nonzero yaw moment. The constraint above makes that command impossible. Configuration (b) therefore cannot track an arbitrary independently prescribed trajectory (x, y, z, Ψ).

2.3 Deliverable 2B: Compensation for quadratic body drag

Let $R \in SO(3)$ map body vectors to world coordinates and let v be world velocity. The drag law in the assignment becomes

$$v^b = R^T v, \qquad D = \operatorname{diag}(0.1, 0.1, 0.2), \qquad F_D^b = -D\Big(v^b \odot |v^b|\Big), \qquad F_D^w = R F_D^b, \tag{7}$$

where $\odot$ denotes componentwise multiplication. Its sign is physically consistent because

$$(v^b)^T F_D^b = -0.1|v_x^b|^3 - 0.1|v_y^b|^3 - 0.2|v_z^b|^3 \leq 0. \tag{8}$$

With $e_3 = (0, 0, 1)^T$ and upward z, translation obeys

$$m\dot{v} = -mge_3 + fRe_3 + F_D^w. \tag{9}$$

Define $e_x = x - x_d$ and $e_v = v - v_d$. Subtract the estimated world-frame drag from the commanded force:

$$\boxed{A = -k_x e_x - k_v e_v + mge_3 + m\ddot{x}_d - \widehat{F}_D^w.} \tag{10}$$

Compute the estimate from measured velocity and attitude, then use this same A for both the desired thrust direction and thrust magnitude. If the attitude follows that direction

and the drag estimate is exact, substitution into the dynamics gives $m\dot{e}_v = -k_x e_x - k_v e_v$. Otherwise, the remaining terms include attitude tracking error and $F_D^w - \widehat{F}_D^w$.

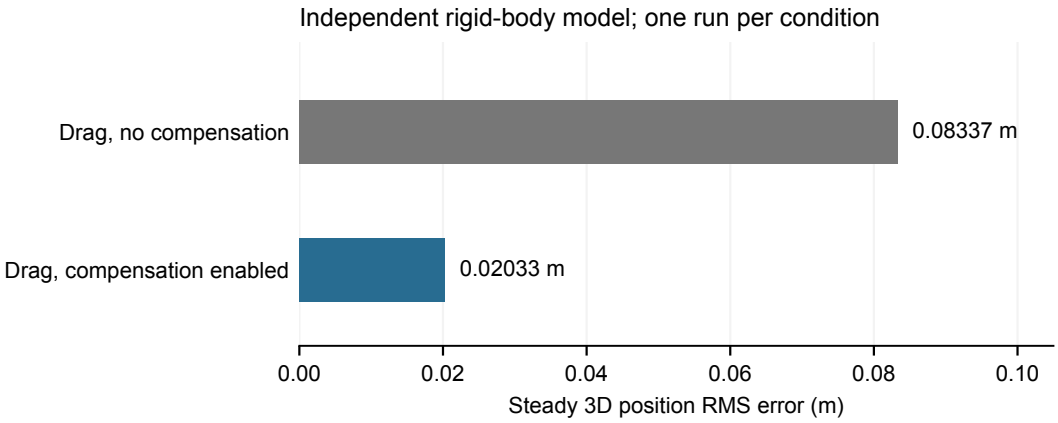


Figure 2: Recorded compensation comparison in the independent rigid-body model. Each bar is one deterministic run with a 1 ms integration step and the prescribed quadratic drag. RMS error is evaluated after two circular periods. The gains are $(k_x, k_v, k_R, k_\Omega) = (6, 4.7, 64, 16)$. These values come from the retained numerical validation log; no uncertainty interval is estimated.

The numerical comparison in Figure 2 reduced the recorded RMS error from 0.08337 m to 0.02033 m, a reduction of 75.6%. This result supports the compensation mechanism under the specified plant model. The official TESSE flight reported below used compensation disabled, so the two evaluations address different conditions.

3 Team Work

3.1 Deliverable 3: ROS control architecture

The completed controller closes the loop between the trajectory publisher and the official TESSE simulator through the ROS bridge (Figure 3). Its three interfaces are `/desired_state`, `/current_state` and `/rotor_speed_cmds`. They carry `MultiDOFJointTrajectoryPoint`, `Odometry` and `Actuators` messages, respectively.

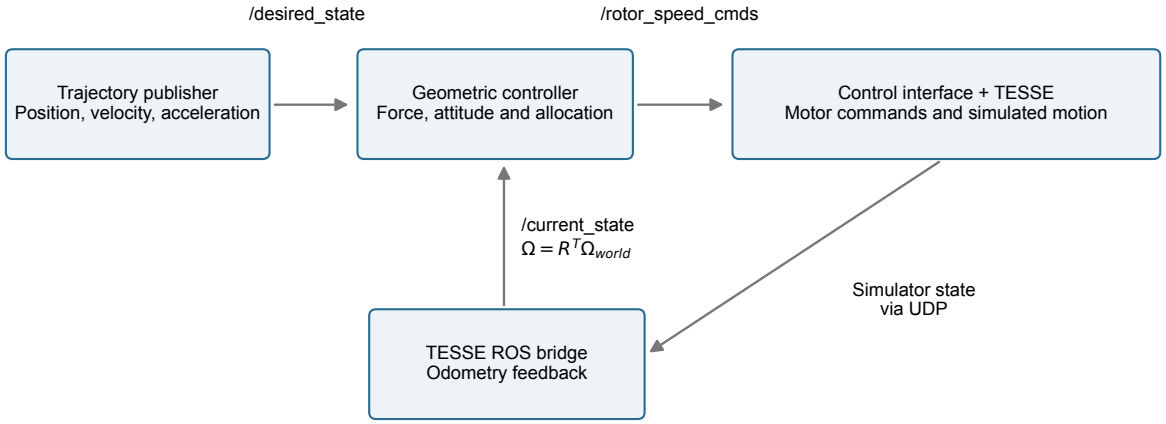


Figure 3: Implemented control loop. The desired state supplies position, velocity, acceleration and yaw. The controller converts the received world-frame angular velocity to the body frame, computes thrust and moments, and allocates them to four rotor speeds. The bridge returns the simulated state.

The implementation checks message-array sizes, finite values and quaternion norms before updating the control state. It stores the most recent valid reference and measurement. The final configuration suppresses command publication when either input is more than 1 s old. The timer is configured at 200 Hz; this is a requested control rate, while transport and simulator scheduling determine the actual message rate.

The recorded run used ROS Noetic in an ARM64 container, with the unmodified x86_64 Unity/TESSE executable running through Box64 0.3.6. The retained build log reports successful compilation of the controller and its dependency packages. The model constants used by the controller are $m = 1$ kg, $g = 9.81\ \mathrm{m\,s^{-2}}$, $d = 0.3$ m, $c_f = 10^{-3}$ and $c_d = 10^{-5}$, with angular speed in $\mathrm{rad\,s^{-1}}$. It retains the skeleton's $J = I_3$ inertia approximation.

3.2 Geometric feedback and rotor allocation

For the official flight, the commanded force is Equation (10) without the drag term. The reference heading is $b_{1c} = (\cos\psi_d, \sin\psi_d, 0)^T$. The desired attitude follows from

$$b_{3d} = \frac{A}{\|A\|}, \qquad b_{2d} = \frac{b_{3d} \times b_{1c}}{\|b_{3d} \times b_{1c}\|}, \qquad R_d = [\, b_{2d} \times b_{3d} \quad b_{2d} \quad b_{3d} \,]. \tag{11}$$

The positive b_{3d} sign matches the upward thrust convention. The code handles a near-zero force with a vertical direction and uses a perpendicular heading when the cross product

becomes degenerate.

The attitude error, thrust and moment implemented in the controller are

$$e_R = \tfrac{1}{2}\left(R_d^T R - R^T R_d\right)^\vee, \tag{12}$$

$$f = A \cdot Re_3, \tag{13}$$

$$M = -k_R e_R - k_\Omega \Omega + \Omega \times (J\Omega). \tag{14}$$

The vee map returns $(S_{32}, S_{13}, S_{21})^T$ for a skew-symmetric matrix S. The implementation uses the simplifying choice $\Omega_d = \dot{\Omega}_d = 0$ in the attitude feedback law. Although the reference yaw changes with time, its rate is not included as feedforward. The measured angular velocity is transformed as $\Omega = R^T \Omega_{\text{world}}$ before applying Equation (14).

The simulator uses a different rotor numbering from Equation (3). Its positions are $(a, a, 0)$, $(-a, a, 0)$, $(-a, -a, 0)$ and $(a, -a, 0)$ for rotors 1 to 4. Defining $s_i = \omega_i|\omega_i|$, the implemented map is

$$\begin{bmatrix} f \\ M_x \\ M_y \\ M_z \end{bmatrix} = F_{\text{sim}} s, \qquad F_{\text{sim}} = \begin{bmatrix} c_f & c_f & c_f & c_f \\ ac_f & ac_f & -ac_f & -ac_f \\ -ac_f & ac_f & ac_f & -ac_f \\ c_d & -c_d & c_d & -c_d \end{bmatrix}. \tag{15}$$

The inverse is computed once. Each control update obtains $s = F_{\text{sim}}^{-1} w$ and publishes $\omega_i = \text{sign}(s_i)\sqrt{|s_i|}$ in $\text{rad}\,\text{s}^{-1}$. This signed square root preserves the simulator's reversible input convention.

3.3 Reference trajectory and evaluation

The circular reference is

$$x_d(t) = (5\sin(t/2),\, 5\cos(t/2),\, 2)^T, \qquad \psi_d(t) = -t/2, \tag{16}$$

$$v_d(t) = (2.5\cos(t/2),\, -2.5\sin(t/2),\, 0)^T, \tag{17}$$

$$a_d(t) = (-1.25\sin(t/2),\, -1.25\cos(t/2),\, 0)^T. \tag{18}$$

It has radius 5 m, speed $2.5\,\text{m}\,\text{s}^{-1}$ and period $T = 4\pi \approx 12.566$ s. The retained experiment sequence includes a 25 s stationary test, two circular gain trials and the final recorded circular flight.

The final CSV contains 2,752 state samples over 55.02 s of reference time. At each state callback, the logger pairs the state with the latest received reference. Repeated reference timestamps are therefore retained. The established steady-window rule is $t \geq 2T = 25.1327$ s, which selects 1,495 samples with reference times from 25.18 to 55.02 s. The first 1,257 samples remain visible in the full-run plots.

For the selected sample set $\mathcal{S}$, the position and radius errors are

$$E_p = \sqrt{\frac{1}{|\mathcal{S}|}\sum_{i\in\mathcal{S}} \|x_i - x_{d,i}\|^2}, \qquad E_r = \sqrt{\frac{1}{|\mathcal{S}|}\sum_{i\in\mathcal{S}} \left(\sqrt{x_i^2 + y_i^2} - 5\right)^2}. \tag{19}$$

Actual turns are counted using the unwrapped phase $\theta_i = \text{unwrap}(\text{atan2}(x_i, y_i))$, consistent with the sine–cosine reference. The change in phase divided by 2π measures completed motion independently of the elapsed reference periods. All statistics describe time samples from one flight.

3.4 Recorded flight and parameter selection

The vehicle completed 2.376 observed turns in the final steady window. The trajectory plot in Figure 4 shows close radial tracking after the initial transient, while the height trace remains below the 2 m reference. The full-run error curve includes the initial mismatch as the moving reference advances around the circle.

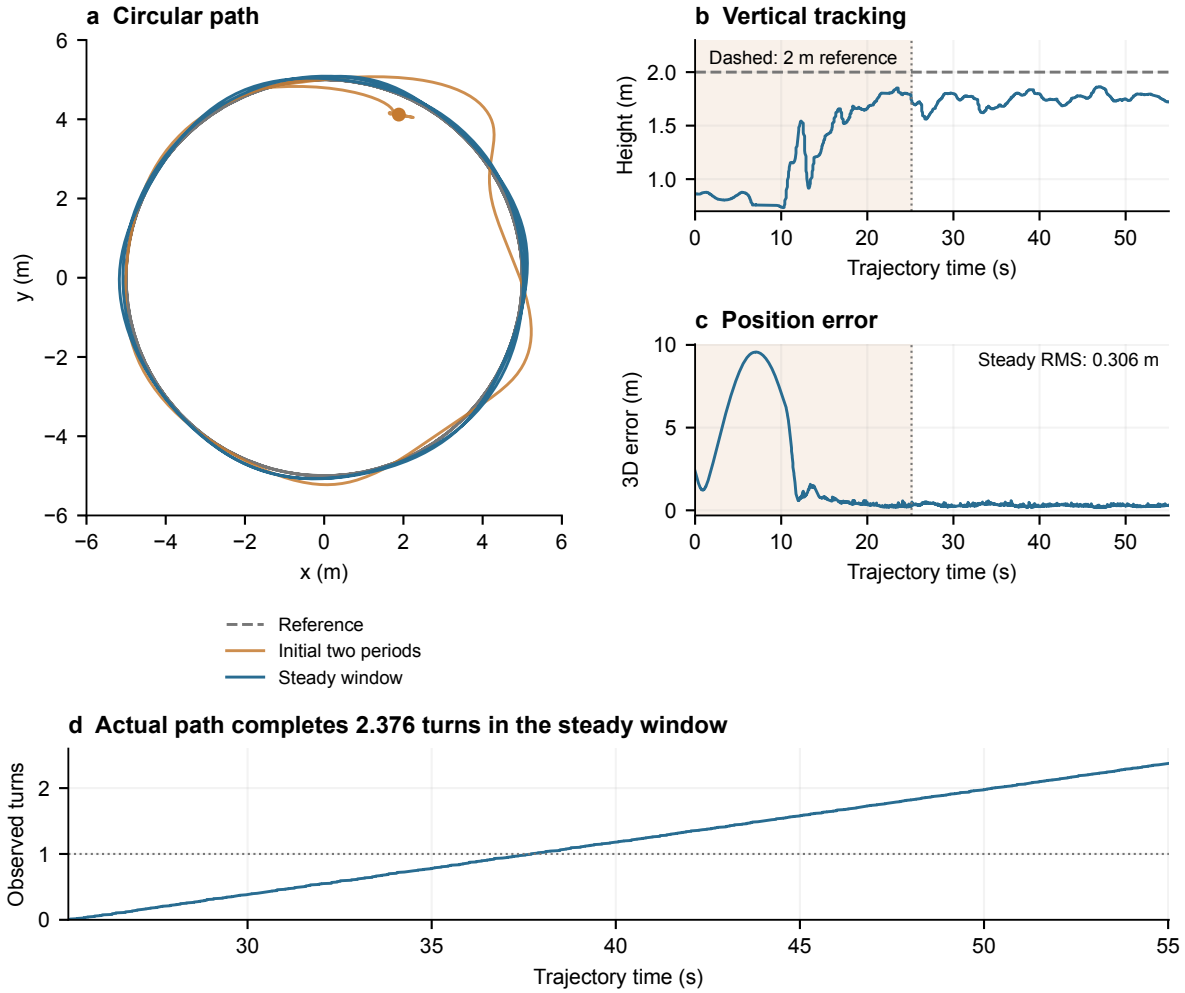


Figure 4: Official TESSE flight, using all 2,752 recorded samples. (a) Top view, with the initial two periods in orange and the steady window in blue. The dot marks the first measured position. (b) Measured height and target. (c) Three-dimensional position-error norm. Shading in (b,c) identifies the interval excluded from steady statistics. (d) Cumulative turns from measured position during the 1,495-sample steady window. The phase increases by 2.3756 turns.

The spatial view in Figure 5 separates the circular geometry from the vertical offset. Figure 6 resolves the same steady-window error into its three coordinate components.

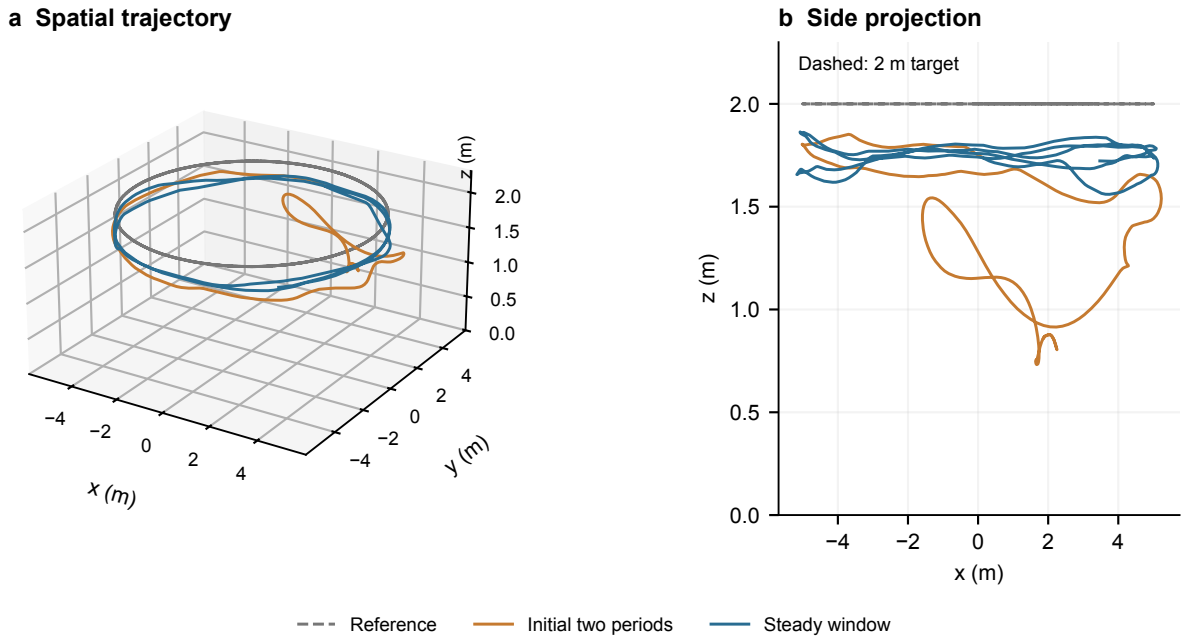


Figure 5: Spatial views of the official flight. (a) Three-dimensional trajectory and (b) projection onto the x–z plane. All 2,752 samples are retained, with the initial two periods in orange and the steady window in blue. The reference lies at $z = 2$ m. Axis scales differ to make the vertical separation readable.

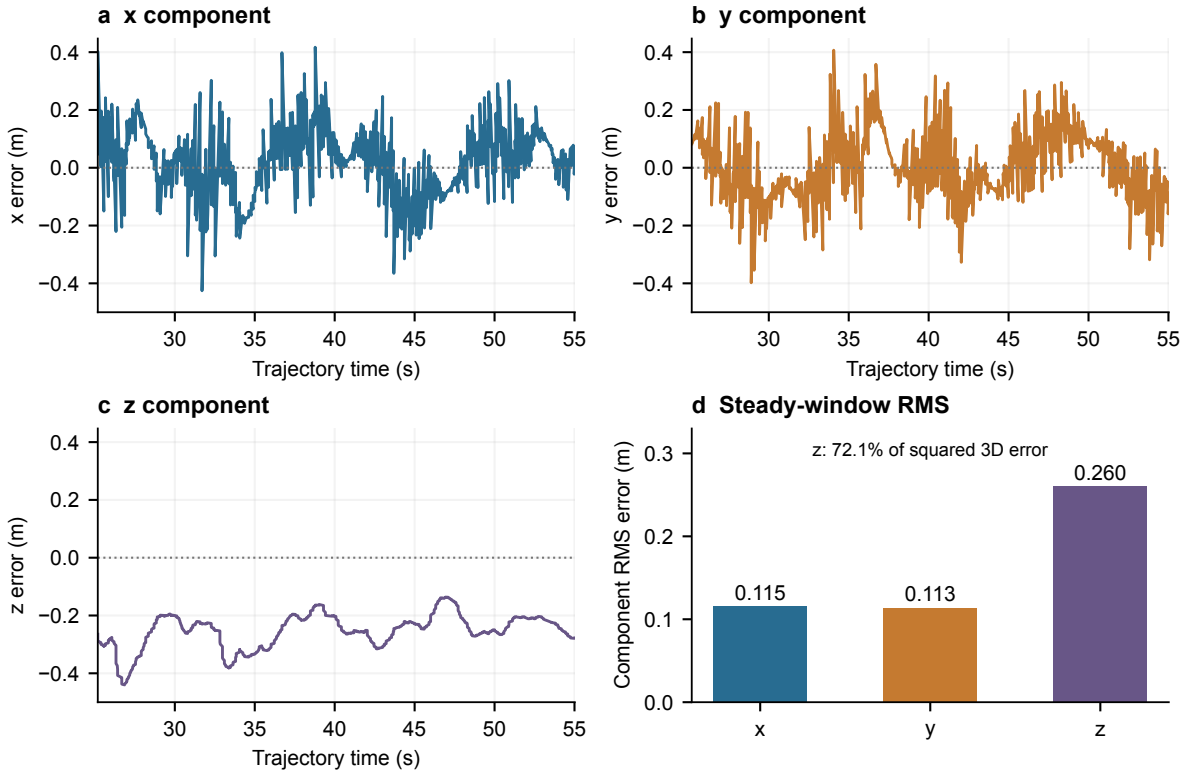


Figure 6: Steady-window position errors from 1,495 state samples. (a–c) Signed errors $x - x_d$, $y - y_d$ and $z - z_d$, shown with common vertical limits. (d) RMS error of each component. The vertical component accounts for 72.1% of mean squared three-dimensional error. These are time samples from one flight; the bars show RMS magnitudes, with no replicate uncertainty estimate.

Table 1: Final official flight metrics, evaluated after two reference periods.

Quantity	Recorded value
Three-dimensional position RMS error	0.3063 m
Maximum position-error norm	0.5845 m
Radial RMS error	0.0829 m
Component RMS errors (x, y, z)	(0.1153, 0.1135, 0.2601) m
Mean height error	−0.2532 m
Measured height range	1.5598 to 1.8638 m
Actual turns in the steady window	2.3756

The selected gains were $(k_x, k_v, k_R, k_\Omega) = (2, 2.4, 1.8, 0.7)$. The retained gain trials in Table 2 support this choice: increasing several gains together produced a larger error in the tested condition. Each gain setting was recorded once, so the comparison describes these trials rather than a statistical ranking across repeated flights.

Table 2: Recorded circular trials. Trial metrics use the final approximately 20 s; the final recording uses the fixed two-period exclusion.

Run	$(k_x, k_v, k_R, k_\Omega)$	Position RMS error (m)
Initial circular trial	$(2, 2.4, 1.8, 0.7)$	0.2710
Higher-gain trial	$(4, 3, 2.4, 0.6)$	0.8970
Final recorded flight	$(2, 2.4, 1.8, 0.7)$	0.3063

Figure 7 shows the error histories behind the trial summaries. The two trials began from different states, so their initial transients are shown explicitly. Figure 8 presents the signed rotor commands for the final flight, connecting the allocation law to the recorded control output.

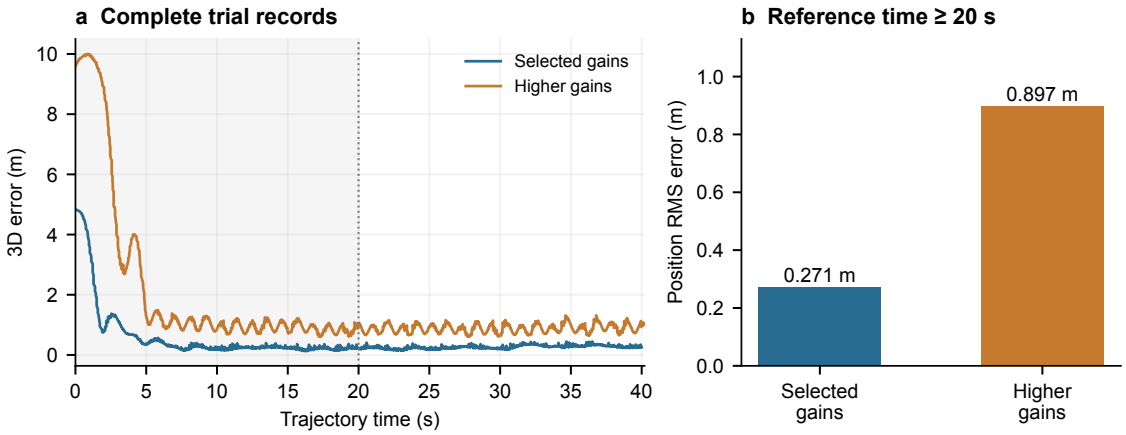


Figure 7: Official TESSE gain trials, using the settings in Table 2. (a) Complete error traces; shading marks the first 20 s. (b) RMS error after that interval. Each setting has one recorded trial, with different starting states.

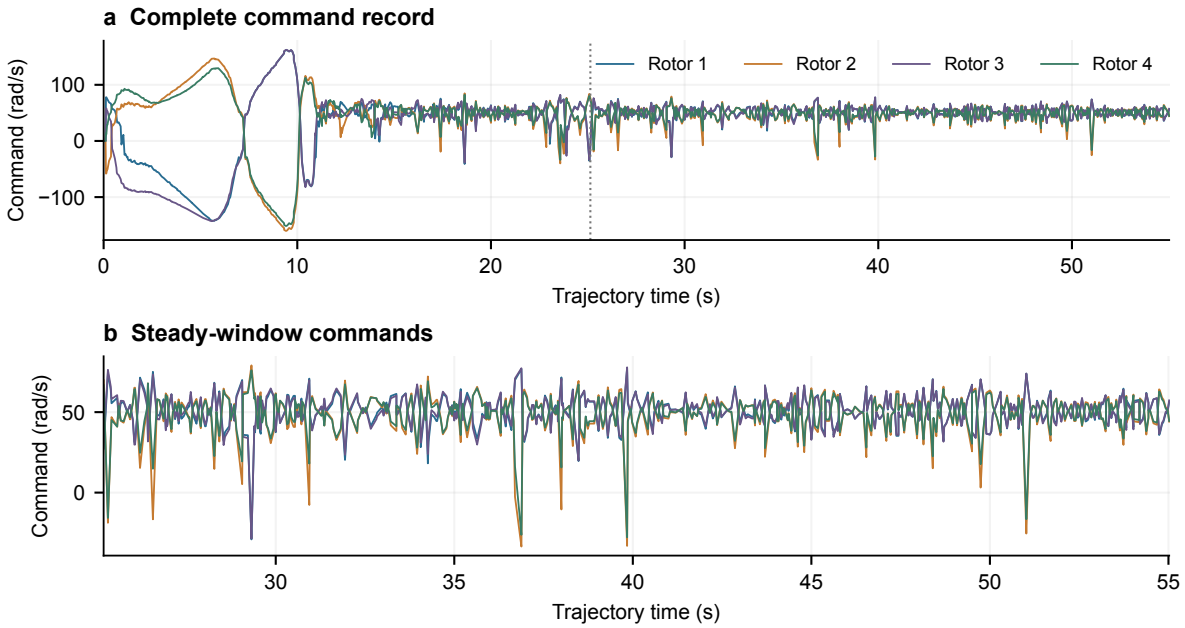


Figure 8: Four rotor-speed command channels in the final official flight. (a) Full record, with a dotted line at the start of the steady window. (b) Enlarged steady-window view. Each value is the latest command logged at a state callback, in rad s^{-1}. Negative values retain the signed allocation convention. These channels record commanded speeds, not measured rotor motion.

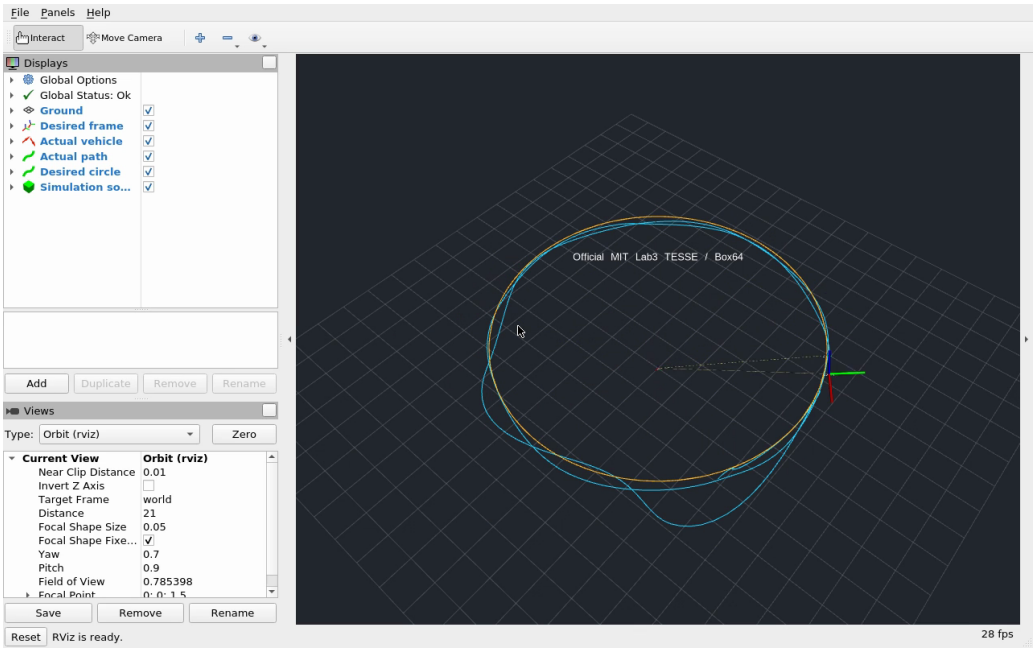


Figure 9: Unmodified frame from the retained RViz recording of the official TESSE run. The cyan measured path and orange reference circle show the flight in the world frame. The on-screen source label identifies the official simulator. The retained local excerpt lasts 27.125 s at 8 fps and retains the original playback speed.

The 27.125 s excerpt begins 30 s into the 57.125 s full recording. The retained bag contains 10,487 messages over 56.4 s, including state, reference and motor commands. These durations refer to video or bag time and need not equal the CSV's 55.02 s reference-time

span. The recording and its local path are retained with the deliverables; no public video link has been created.

4 Discussion

The allocation proof explains a structural requirement for tracking position and yaw independently. The original layout provides four independent wrench directions. In the modified layout, yaw moment is tied to roll moment, so feedback tuning cannot recover the missing actuation direction. The simulator matrix also shows why rotor numbering and yaw signs must be checked separately when implementing the controller.

The official flight demonstrates repeated circular motion with bounded error in the recorded window. Its 0.0829 m radial RMS error is about 1.66% of the reference radius. Height accounts for 72.1% of the mean squared three-dimensional position error, with a mean height offset of -0.2532 m. Thus the planar circle and the three-dimensional accuracy convey different aspects of the result. Evaluating both prevents the top view from obscuring vertical behaviour.

Several implementation choices are relevant to this offset and the gain sensitivity. The moment law omits desired angular-rate feedforward, the controller uses the skeleton inertia approximation, and simulator scheduling holds motor commands between updates. The available trials change multiple gains simultaneously and do not isolate those mechanisms. A direct follow-up would compare vertical error with one model or feedforward change at a time, using the same trajectory and evaluation window.

The drag calculation has a narrower, well-defined purpose: it cancels a specified force model. The independent numerical runs support that mechanism, reducing RMS error by 75.6% when the plant includes the same drag law. Applying the term to TESSE would require checking that its force model is appropriate. The official flight used the uncompensated controller, and its measured accuracy remains the basis for the simulator result.

Finally, the error metric uses the most recent received reference at each state callback. It therefore includes asynchronous reference updates as well as vehicle tracking behaviour. The 1,495 steady samples are correlated observations from one run; they quantify that trajectory and do not establish repeatability across runs or initial conditions.

5 Conclusions

The lab links quaternion conversion and rotor allocation to a working ROS geometric controller. The rank proof identifies the loss of independent yaw authority after reversing two rotor configurations, while the drag model gives a consistent force correction. The official TESSE recording contains 2.376 steady-window turns with a position RMS error of 0.3063 m. Radial tracking and the measured height offset identify the achieved circular motion and the principal direction for further control refinement.

References

[1] MIT VNAV. Lab 3: Exercises. `https://vnav.mit.edu/labs_2023/lab3/exercises.html`, 2023. Accessed 10 October 2026.

[2] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *49th IEEE Conference on Decision and Control*, pages 5420–5425, 2010. doi: 10.1109/CDC.2010.5717652.

[3] ROS geometry2 contributors. tf2_geometry_msgs: Quaternion message conversion. `https://github.com/ros/geometry2/blob/noetic-devel/tf2_geometry_msgs/include/tf2_geometry_msgs/tf2_geometry_msgs.h`, . ROS Noetic source, accessed 10 October 2026.

[4] ROS geometry2 contributors. tf2: Yaw extraction utilities. `https://github.com/ros/geometry2/blob/noetic-devel/tf2/include/tf2/utils.h`, . ROS Noetic source, accessed 10 October 2026.

[5] ROS geometry2 contributors. tf2_eigen: Quaternion conversion to eigen. `https://github.com/ros/geometry2/blob/noetic-devel/tf2_eigen/include/tf2_eigen/tf2_eigen.h`, . ROS Noetic source, accessed 10 October 2026.